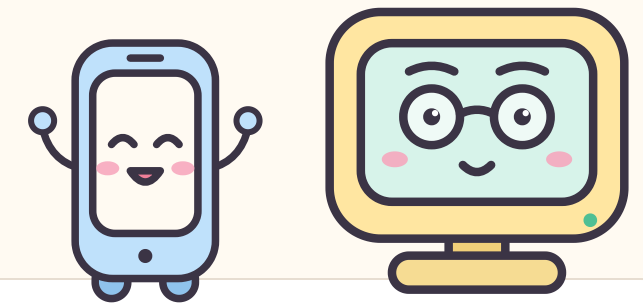




第 18 集 · 约 19 分钟

网页①：协议客户端

浏览器这一边，怎么配对、怎么发加密请求

18

从零写出 LanShare

这一集你会学到

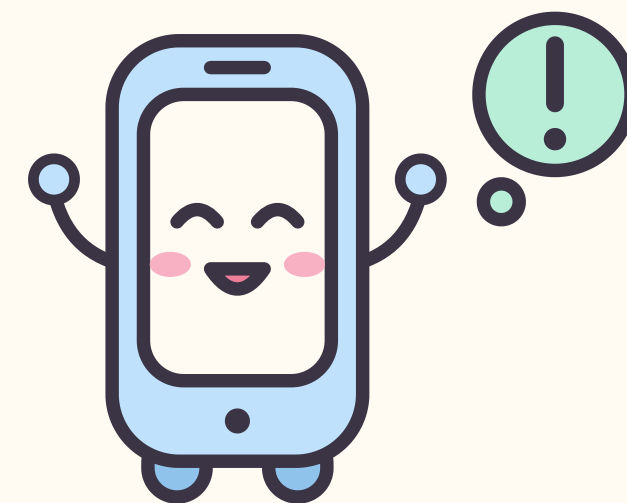
1. 扫码配对和口令配对，在浏览器这一边怎么写
2. 刷新页面不用重扫：把 K 记在 localStorage，用新的 cid 重新握手
3. send：计数器、请求头、附加数据，一个加密请求的全部细节
4. 分块上传和下载：3 个块同时在路上，失败自动重试
5. 互通测试：Node 里的 proto.js 对真实启动的 exe 走完整流程



先想一个问题

小手机扫码进来，聊了几句，然后手一滑，刷新了页面。它还要再扫一次码吗？

不用。K 记在浏览器的 localStorage 里，刷新后拿它换一个新的 cid，重新握手。





小手机要学会的四件事



发生了什么

窃听器只看到证明和 SPAKE2 消息，算不出钥匙

窃听器只看到会话号、计数器、接口路径和密文长度



分隔线、常量和错误类型

proto.js crates/lanshare/web/

第 111-130 行 • 共 130 行

```
111 // 协议客户端: 配对 + 加密请求 + 分块传输
112 // -----
113
114 const CHUNK = 1 << 20;
115 const STORE_KEY = "lanshare";
116
117 export class Unauthorized extends Error {}
118 export class WrongPin extends Error {}
119 export class HttpError extends Error {
120     constructor(status) {
121         super("HTTP " + status);
122         this.status = status;
123     }
124 }
125 export class ServerError extends Error {
126     constructor(code) {
127         super(code);
128         this.code = code;
129     }
130 }
```



编码小工具和密钥解析

proto.js crates/lanshare/web/

第 145–164 行 • 共 164 行

```

145     return out;
146 }
147
148 const b64urlEncode = (u8) => b64Encode(u8).replace(/\+/g, "-").replace(/\//g, "_").replace(/=+$/, "");
149
150 /** 从 location.hash (“#<base64url>”) 取出 32 字节密钥; 不是合法密钥返回 null。 */
151 export function keyFromFragment(hash) {
152     const raw = (hash || "").replace(/^#/g, "");
153     if (!/^[A-Za-z0-9_-]{43}$/g.test(raw)) return null;
154     try {
155         const key = b64Decode(raw.replace(/-/g, "+").replace(/_/g, "/") + "=");
156         return key.length === 32 ? key : null;
157     } catch {
158         return null;
159     }
160 }
161
162 const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
163 const utf8 = new TextEncoder();
164 const fromUtf8 = new TextDecoder();

```




客户端类：构造和服务标识

proto.js crates/lanshare/web/

第 165–184 行 • 共 185 行

```

165
166 export class LanShareClient {
167   constructor(crypto, { base = "", storage = globalThis.localStorage, fetchImpl } = {}) {
168     this.crypto = crypto;
169     this.base = base;
170     this.storage = storage;
171     this.fetch = fetchImpl || ((...a) => globalThis.fetch(...a));
172     this.keys = null;
173     this.cid = null;
174     this.ctr = 0;
175     this.serverIdCache = null;
176   }
177
178   async serverId() {
179     if (!this.serverIdCache) {
180       const res = await this.fetch(this.base + "/api/server", { cache: "no-store" });
181       if (!res.ok) throw new HttpError(res.status);
182       this.serverIdCache = (await res.json()).server_id;
183     }
184     return this.serverIdCache;

```



记住密钥、启用会话

● ● ●

proto.js crates/lanshare/web/

第 186-205 行 • 共 205 行

```
186
187 #remember(key) {
188     try {
189         this.storage?.setItem(STORE_KEY, JSON.stringify({ server_id: this.serverIdCache, key: b64urlEncode(key) }));
190     } catch { /* 隐私模式等存不了也不影响本次使用 */ }
191 }
192
193 forget() {
194     try { this.storage?.removeItem(STORE_KEY); } catch { /* 忽略 */ }
195 }
196
197 #useSession(cid, sessionSecret) {
198     this.cid = cid;
199     this.keys = this.crypto.channelKeys(sessionSecret);
200     this.ctr = 0;
201 }
202
203 get paired() {
204     return this.keys !== null;
205 }
```



扫码配对

proto.js crates/lanshare/web/

第 201-220 行 · 共 220 行

```

201     }
202
203     get paired() {
204         return this.keys !== null;
205     }
206
207     /** 扫码配对: 证明知道二维码里的密钥 K, 派生本会话的密钥。 */
208     async pairWithKey(key) {
209         await this.serverId();
210         const cid = this.crypto.randomBytes(16);
211         const res = await this.fetch(this.base + "/api/hello", {
212             method: "POST",
213             headers: { "Content-Type": "application/json" },
214             body: JSON.stringify({ cid: hex(cid), tag: hex(this.crypto.helloTag(key, cid)) }),
215         });
216         if (res.status === 403) throw new Unauthorized();
217         if (!res.ok) throw new HttpError(res.status);
218         this.#useSession(cid, this.crypto.sessionFromKey(key, cid));
219         this.#remember(key);
220     }

```




口令配对

proto.js crates/lanshare/web/

第 221-240 行 • 共 247 行

```
221
222  /** 口令配对 (SPAKE2) : 口令不经过网络; 成功后经加密信道拿到 K 并记住。 */
223  async pairWithPin(pin) {
224      const serverId = await this.serverId();
225      const cid = this.crypto.randomBytes(16);
226      const { handle, msg } = this.crypto.pakeStart(pin, serverId);
227      let res = await this.fetch(this.base + "/api/pake/start", {
228          method: "POST",
229          headers: { "Content-Type": "application/json" },
230          body: JSON.stringify({ cid: hex(cid), msg: b64Encode(msg) }),
231      });
232      if (!res.ok) throw new HttpError(res.status);
233      const reply = await res.json();
234      const pakeKey = this.crypto.pakeFinish(handle, b64Decode(reply.msg));
235      if (!pakeKey || !this.crypto.verifyConfirm(pakeKey, "server", cid, unhex(reply.confirm))) throw new WrongPin();
236      res = await this.fetch(this.base + "/api/pake/finish", {
237          method: "POST",
238          headers: { "Content-Type": "application/json" },
239          body: JSON.stringify({ cid: hex(cid), confirm: hex(this.crypto.confirmTag(pakeKey, "client", cid)) }),
240      });
```



刷新后免配对：resume

proto.js crates/lanshare/web/

第 244–263 行 • 共 263 行

```
244     const info = await this.rpc({ op: "info" });
245     const key = keyFromFragment("#" + info.key);
246     if (key) this.#remember(key);
247 }
248
249 /** 用以前存下的密钥重新握手（换新的会话 id）；没存过或不是这次运行的返回 false。 */
250 async resume() {
251     let saved = null;
252     try { saved = JSON.parse(this.storage?.getItem(STORE_KEY) || "null"); } catch { /* 忽略 */ }
253     if (!saved || saved.server_id !== (await this.serverId())) return false;
254     const key = keyFromFragment("#" + saved.key);
255     if (!key) return false;
256     try {
257         await this.pairWithKey(key);
258         return true;
259     } catch (e) {
260         if (e instanceof Unauthorized) this.forget();
261         return false;
262     }
263 }
```



一个加密请求：send

proto.js crates/lanshare/web/

第 264-283 行 • 共 290 行

```
264
265  /** 发一个加密请求, 返回解密后的字节。 */
266  async send(path, plain) {
267      if (!this.keys) throw new Unauthorized();
268      const ctr = ++this.ctr;
269      const body = this.crypto.seal(this.keys.c2s, ctr, utf8.encode("lanshare/2 req " + path), plain);
270      const res = await this.fetch(this.base + path, {
271          method: "POST",
272          cache: "no-store",
273          headers: { "X-LS-Sid": hex(this.cid), "X-LS-Ctr": String(ctr), "Content-Type": "application/octet-stream" },
274          body,
275      });
276      if (res.status === 401) {
277          this.keys = null;
278          throw new Unauthorized();
279      }
280      if (!res.ok) throw new HttpError(res.status);
281      const sealed = new Uint8Array(await res.arrayBuffer());
282      const out = this.crypto.open(this.keys.s2c, ctr, utf8.encode("lanshare/2 resp " + path), sealed);
283      if (!out) throw new Error("响应解密失败");
```



业务错误和重试

● ● ● proto.js crates/lanshare/web/ 第 289-308 行 • 共 308 行

```
289     return JSON.parse(fromUtf8.decode(out));
290 }
291
292 async #rpcOk(request) {
293     const r = await this.rpc(request);
294     if (!r.ok) throw new ServerError(r.error);
295     return r;
296 }
297
298 /** 网络抖动重试 3 次；未登录、业务错误不重试。 */
299 async #retry(fn) {
300     for (let attempt = 0; ; attempt++) {
301         try {
302             return await fn();
303         } catch (e) {
304             if (e instanceof Unauthorized || e instanceof ServerError || attempt >= 3) throw e;
305             await sleep(400 * (attempt + 1));
306         }
307     }
308 }
```



分块上传

proto.js crates/lanshare/web/

第 309–328 行 • 共 338 行

```
309
310  /** 分块并发上传，返回服务端保存的文件名。onProgress(已发送字节数)。 */
311  async uploadFile(file, { onProgress = () => {}, parallel = 3 } = {}) {
312    const { upload_id: id } = await this.#rpcOk({ op: "upload_begin", name: file.name, size: file.size });
313    const total = Math.ceil(file.size / CHUNK);
314    let next = 0;
315    let sent = 0;
316    const worker = async () => {
317      while (next < total) {
318        const index = next++;
319        const end = Math.min(file.size, (index + 1) * CHUNK);
320        const piece = new Uint8Array(await file.slice(index * CHUNK, end).arrayBuffer());
321        await this.#retry(async () => {
322          const r = JSON.parse(fromUtf8.decode(await this.send(`/api/up/${id}/${index}`, piece)));
323          if (!r.ok) throw new ServerError(r.error);
324        });
325        sent += piece.length;
326        onProgress(sent);
327      }
328    };
```




分块下载



proto.js

crates/lanshare/web/

第 339–358 行 • 共 362 行

```
339
340  /** 分块并发下载, 返回 Blob。`entry` 是列表里的一项 {id, name, size, mtime}。 */
341  async downloadFile(entry, { onProgress = () => {}, parallel = 3 } = {}) {
342      const total = Math.max(1, Math.ceil(entry.size / CHUNK));
343      const parts = new Array(total);
344      const expect = utf8.encode(JSON.stringify({ size: entry.size, mtime: entry.mtime }));
345      let next = 0;
346      let received = 0;
347      const worker = async () => {
348          while (next < total) {
349              const index = next++;
350              parts[index] = await this.#retry(async () => {
351                  const out = await this.send(`/api/down/${entry.id}/${index}`, expect);
352                  if (out[0] === 1) throw new ServerError(JSON.parse(fromUtf8.decode(out.subarray(1))).error);
353                  return out.subarray(1);
354              });
355              received += parts[index].length;
356              onProgress(received);
357          }
358      };
```



互通测试：准备

interop.test.mjs web-tests/ 第 24-43 行 · 共 43 行

```
24
25 let proc, info, base, crypto, work;
26
27 before(async () => {
28   work = mkdtempSync(join(tmpdir(), "lanshare-interop-"));
29   const infoPath = join(work, "info.json");
30   proc = spawn(EXE, ["--no-tray", "--no-browser", "--port", "0", "--dir", join(work, "share"), "--info-file", infoPath],
31     { stdio: "ignore", env: { ...process.env, LOCALAPPDATA: work } });
32   for (let i = 0; i < 200 && !existsSync(infoPath); i++) await new Promise((r) => setTimeout(r, 50));
33   info = JSON.parse(readFileSync(infoPath, "utf8"));
34   base = `http://127.0.0.1:${info.port}`;
35   crypto = await loadCrypto(readFileSync(WASM), (b) => webcrypto.getRandomValues(b));
36 });
37
38 after(() => {
39   proc?.kill();
40   try { rmSync(work, { recursive: true, force: true }); } catch {}
41 });
42
43 const newClient = (storage = new MemoryStorage()) => new LanShareClient(crypto, { base, storage });
```



互通测试：用例

interop.test.mjs web-tests/

第 111-130 行 • 共 133 行

```
111
112 test("口令错误被识别", async () => {
113     const wrong = info.pin === "000000" ? "111111" : "000000";
114     await assert.rejects(newClient().pairWithPin(wrong), WrongPin);
115 });
116
117 test("没有存过密钥时 resume 返回 false", async () => {
118     assert.equal(await newClient().resume(), false);
119 });
120
121 // 放在最后：会换口令，影响前面用 info.pin 的测试
122 test("本机界面重新开启口令登录：换新口令，新口令能用，旧口令作废", async () => {
123     const local = newClient();
124     await local.pairWithKey(keyFromFragment("#" + info.key));
125     const before = await local.rpc({ op: "info" });
126     assert.equal(before.pin_paused, false);
127     const r = await local.rpc({ op: "pin_resume" });
128     assert.equal(r.ok, true);
129     assert.match(r.pin, /^\\d{6}$/);
130     assert.notEqual(r.pin, before.pin);
```

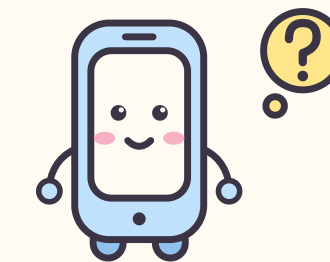



编译，跑互通测试

Windows PowerShell

```
PS D:\code\lanshare> cargo build -p lanshare
   Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
..... (省略 1 行：我的验收环境又编译了一次 lanshare-proto，你跟着写下来不会有这一行)
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 7.32s

PS D:\code\lanshare> node --test web-tests/interop.test.mjs
✓ exe 内嵌的网页资源与仓库里的文件一致（防止打包进旧的 WASM/页面）（107.5796ms）
✓ 二维码片段解析（0.4213ms）
✓ 扫码配对 → 加密 RPC（14.4964ms）
✓ 上传（分块并发）→ 列表 → 下载，内容一致，进度走满（86.8508ms）
✓ 空文件与重名（23.5349ms）
✓ 文字互发（5.8162ms）
✓ 口令配对（PAKE, JS/WASM ↔ Rust）后记住密钥，下次免输入（17.9445ms）
✓ 口令错误被识别（6.3771ms）
✓ 没有存过密钥时 resume 返回 false（0.186ms）
✓ 本机界面重新开启口令登录：换新口令，新口令能用，旧口令作废（16.32ms）
i tests 10
i suites 0
i pass 10
i fail 0
i cancelled 0
```



选择题

刷新页面后，proto.js 为什么不沿用上次的话和计数器，而是用存下的 K 重新握手？

- A 因为 localStorage 存不了数字
- B 刷新、多个标签页时计数器容易撞车，每次换新 cid 重新握手最省心
- C 因为服务端每次请求后都会删掉会话



选择题

刷新页面后，proto.js 为什么不沿用上次的话和计数器，而是用存下的 K 重新握手？

A 因为 localStorage 存不了数字

localStorage 存字符串，数字转一下就能存，不是这个原因。

C 因为服务端每次请求后都会删掉会话

服务端会保留会话，最多 256 个，是客户端自己选择每次重新握手。

B 刷新、多个标签页时计数器容易撞车，每次换新 cid 重新握手最省心

同一个会话里计数器一旦重复，就是同一把钥匙配同一个 nonce，第 3 集说过这会让加密失效；服务端也会把它当成重放拒掉。每次打开页面换一个会话，从根上避开。



选择题

上传时某一块因为网络抖动失败了，重试会不会被防重放窗口拒绝？

- A 会，因为是同一块数据
- B 不会，每次重发都调用 send，用的是一个新的计数器
- C 会，所以失败了只能整个文件重传



选择题

上传时某一块因为网络抖动失败了，重试会不会被防重放窗口拒绝？

A 会，因为是同一块数据

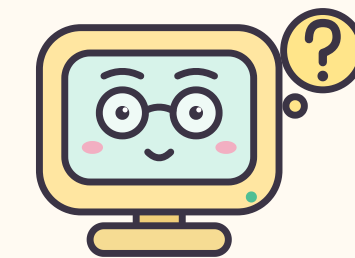
防重放看的是计数器，不是数据内容。

C 会，所以失败了只能整个文件重传

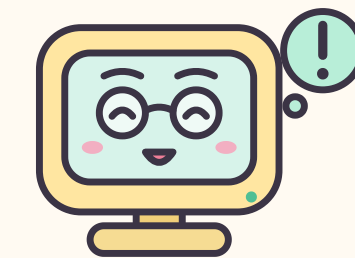
只重传失败的那一块就行。

B 不会，每次重发都调用 send，用的是一个新的计数器

send 每次都先把计数器加 1，重发的那一块带着新的计数器，是一个全新的请求。



互通测试为什么要比对 exe 里内嵌的网页和仓库里的文件？测试已经在用仓库里的 proto.js 了，还不够吗？



互通测试为什么要比对 exe 里内嵌的网页和仓库里的文件？测试已经在用仓库里的 proto.js 了，还不够吗？

测试在 Node 里用的是仓库里的 proto.js，可手机浏览器拿到的，是 exe 里嵌着的那一份。改了 web 下面的文件、忘了重新编译，测试照样全过，用户拿到的却是旧页面、旧 WASM。比对哈希，就能保证“测过的”和“发出去的”是同一份东西。



这一集写完了

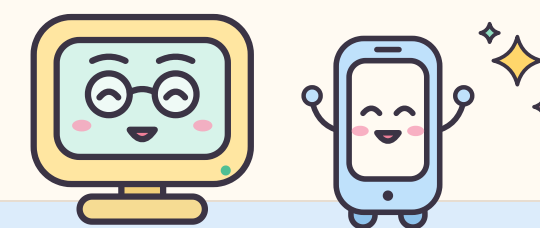
```
lanshare/
├── crates/
│   └── lanshare/
│       ├── src/ 19 个文件
│       ├── tests/ 5 个文件
│       └── web/
│           ├── app.js
│           ├── icon.ico 从仓库拷贝
│           ├── index.html
│           └── lanshare.wasm 从仓库拷贝
```

```
├── proto.js 这一集写的
│   ├── style.css
│   ├── build.rs
│   └── Cargo.toml
├── lanshare-proto/ 13 个文件
├── lanshare-wasm/ 2 个文件
├── web-tests/
│   ├── crypto.test.mjs
│   ├── gen-vectors.mjs
│   └── interop.test.mjs 这一集写的
└── Cargo.toml
```




这一集写了什么

- ✓ 扫码配对、口令配对；口令配对后经加密信道拿到 K
- ✓ K 和电脑标识记在 localStorage，刷新后换新 cid 重新握手
- ✓ send：计数器先加 1，会话号和计数器放请求头，附加数据绑定路径
- ✓ 分块上传下载：3 个工人同时领块，网络抖动自动重试
- ✓ 互通测试：Node 里的 proto.js 对真实的 exe，连内嵌文件的哈希都要比



下一集

全课完结，感谢收看